ЛР1. Декомпозиція Сервісних Систем та Domain Discovery

Тема: Проєктування розподілених сервісних архітектур

Мета: Навчитись розбивати монолітні системи на логічно незалежні сервіси

Ключові аспекти: Domain-Driven Design (DDD): bounded contexts, entities, aggregates, value objects. Шаблони декомпозиції: за бізнес-можливостями, за типами даних, за діями користувача. Приклади використання DDD у мікросервісних архітектурах. Інструменти для виявлення доменів: Event Storming, Context Mapping. Визначення меж сервісу: API surface, автономія, залежності

Що почитати:

https://github.com/ddd-crew/ddd-starter-modelling-process

https://github.com/SAP/curated-resources-for-domain-driven-design/blob/main/knowledgebase/tools/eventstorming.md

https://github.com/kgrzybek/modular-monolith-with-ddd

https://www.youtube.com/watch?v=mLXQIYEwK24

https://www.infoq.com/minibooks/domain-driven-design-quickly/

Короткі теоретичні відомості

Domain driven design

Предметно-орієнтоване проектування (*англ. Domain-driven design*) – це підхід до проектування інформаційної системи, за якого її високорівнева структура та структура коду відображають бізнес галузь (предмет). Так, додаток для автоматизації скорингу міститиме класи **LoanApplication** та **Customer,** а також методи **AcceptOffer** і **Withdraw.**

Предметом (*англ. Domain*) є галузь знань чи діяльності, визначена набором спільних вимог(requirements), термінології, а також функціоналу необхідних програмному застосунку для вирішення поставленої задачі.

Основними поняттями предметно-орієнтованого проектування є:

- **Моделювання** (Model Driven Design) - процес відображення галузі чи системи в цифровому середовищі.

- **Загальна мова** (Ubiquitous Language) - набір однозначної термінології, що описує бізнес-процес/галузь.
- **Патерни**, що в свою чергу діляться на:
    - *Стратегічні* - що ми повинні створити, яке саме програмне забезпечення розробити.
    - *Тактичні* - як ми повинні його створювати.

Стратегічне проектування є складовою моделювання, що дозволяє спроектувати систему, визначивши домени та під-домени, описані універсальною мовою; формалізувати взаємодію між ними.

Основою стратегічного дизайну є ***обмежений контекст*** (англ. Bounded Context), котрий фактично визначає ізольований бізнес процес. Визначення ОК - основна задача ПОП, яка уможливлює коректну декомпозицію системи, забезпечуючи слабку зв'язність між компонентами та сильне зчеплення всередині них (англ. Loose coupling, strong cohesion) [5].

Методології визначення обмеженого контексту:

- Картографування зв'язків сутностей загальної мови
- Event storming
- Mind-maps
- User-stories

Також існує спрощений метод – «підмето-орієнтоване» проєктування. Перелічуємо "підмети" (сутності) системи, а потім додаємо присудки, шоб оцінити зв'язність між ними.

Картографування контекстів

Картографування контекстів - це процес їх визначення та зображення взаємодій між ними. Даний процес дозволяє наочно показати межі контекстів та спроектувати комунікацію між моделями системи, вирішує проблему спільних контекстів.

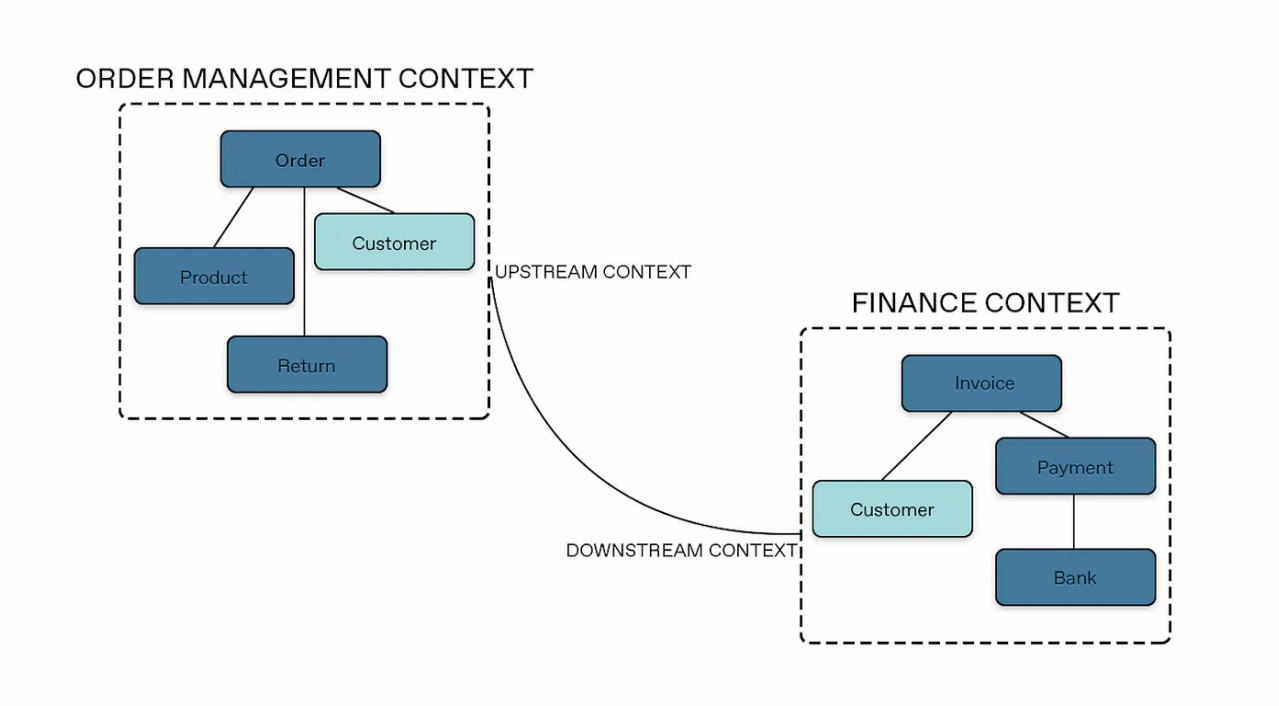


Рис. 2. Картографування контекстів [12]

Класифікуємо шаблони взаємодії відповідно до напрямку залежностей між контекстами.

Рівнозначні (Midway) патерни

Shared Kernel → Частина домену є спільною для різних обмежених контекстів. Таким чином знижується дублікація коду, а отже і вартість підтримки.

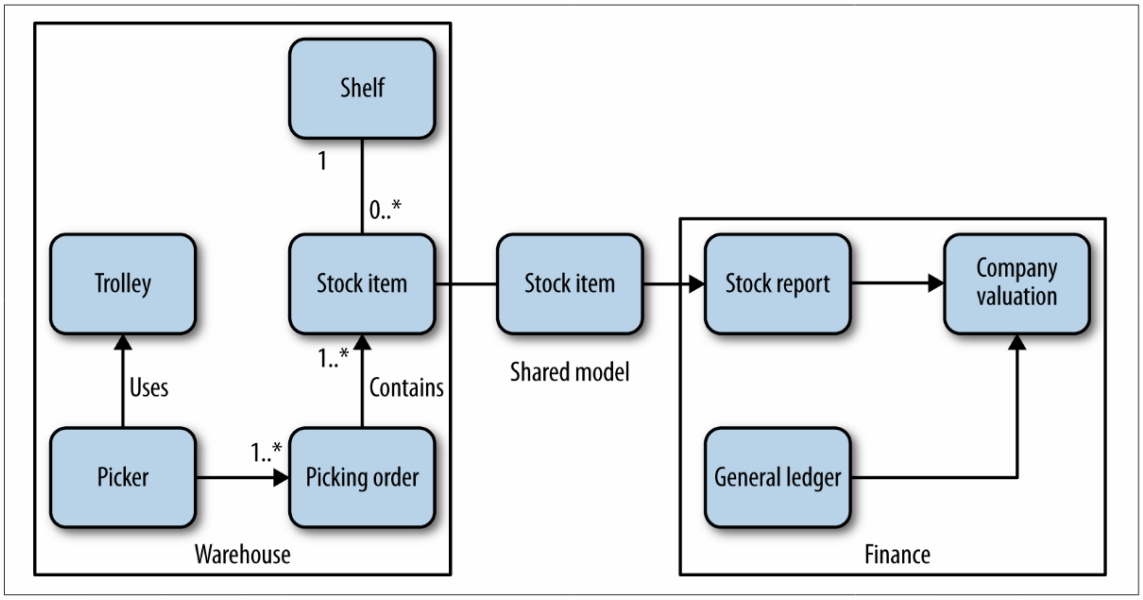


Рис. 3. Шаблон проєктування Shared Kernel [4]

Separate ways → В цьому випадку сервіси не мають між собою спільного контексту.

Висхідні (Upstream) патерни

Open Host Service → Протокол, що надає зовнішнім акторам доступ до сервісів заданої підсистеми. Наприклад, АПІ, яке можуть викликати інші сервіси.

Низхідні (Downstream) патерни

Customer-Supplier → Контекст різних сервісів значною мірою залежить від інших сервісів. Вихідний передає контекст наступному(нижчому) сервісу, а контекст нижчого діє як клієнт, визначаючи вимоги та вносячи зміни у контекст попередніх (верхніх) сервісів. Отже, одним контекстом є клієнт, а інший відіграє роль постачальника. Набір тестів для тестування інтерфейсу між двома контекстами необхідний для забезпечення відповідності обох стандартам інтерфейсу.

Conformist →Це нагадує спільний шаблон ядра, оскільки клієнт використовує модель постачальника та відповідає їй. Різниця полягає в тому, що відносини більш односпрямовані, і клієнт не може вносити зміни в модель постачальника, лише використовуючи її.

Anti-corruption layer → це рівень або сервіс, який існує між внутрішньою моделлю та зовнішньою. Він діє як перекладач між двома контекстами, зберігаючи модель клієнта чистою, узгодженою та вільною від впливу зовнішніх моделей.

На рис.4 наведено схему послідовності вибору шаблонів взаємодії.

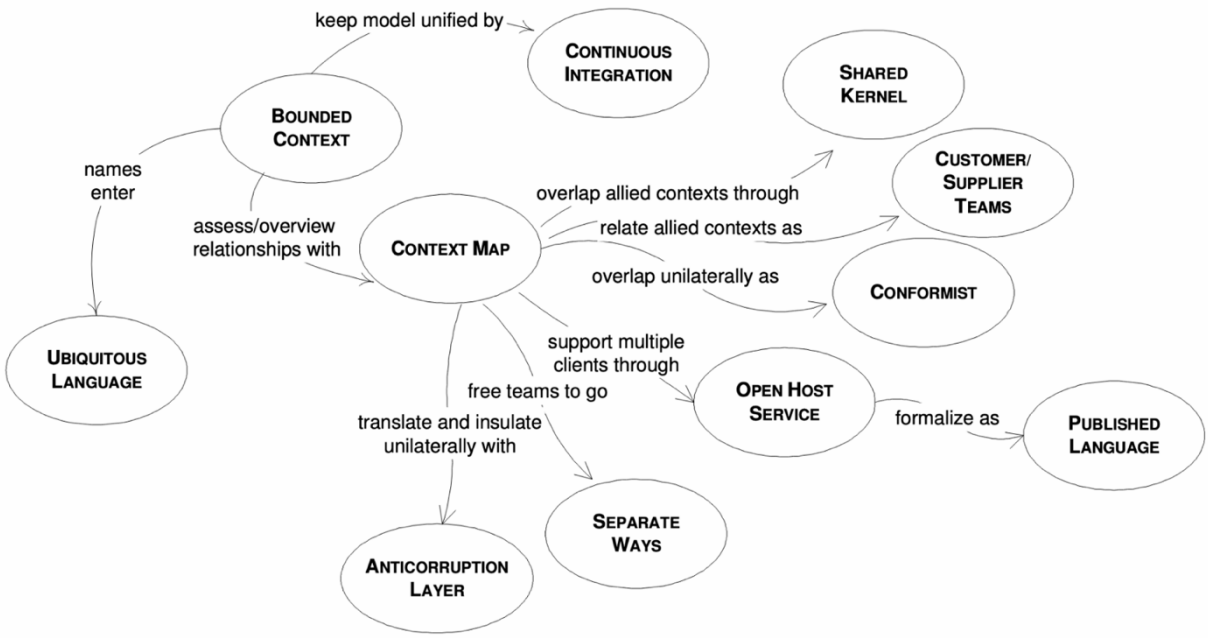


Рис. 4. Стратегічні шаблони проєктування ПОП[4]

Тактичні шаблони

Тактичним дизайном в контексті предметно-орієнтованого проєктування називають набір конструкцій і патернів, призначений для реалізації зв'язків та структури доменних об'єктів в межах моделі.

Доменна модель (domain model): код, який суто описує поведінку, властивості та взаємодію речей, релевантних у бізнес-контексті, який обслуговує програмне забезпечення. При цьому абсолютно ігноруються такі проблеми, як середовище виконання, середовище хостингу тощо.

Тактичний дизайн оперує наступними базовими елементами моделі:

- **Сутності (Entities)** – мають незмінний унікальний ідентифікатор, котрі дозволяють їм мати власний життєвий цикл.
- **Об'єкти-значення (Value objects)** – набори атрибутів, що описують певний аспект предметної області. Мають бути немодифіковуваними (Immutable)
- **Агрегати(Aggregates)** - група пов'язаних об'єктів, котрі є одним цілим в контексті доступу до даних.

- **Події (Domain events)** - повідомлення компонентам системі, що щось відбулося (користувача з таким айді створено)
- **Сервісні компоненти (Services)** – елементи, що інкапсулюють певну бізнес-логіку предметної області, проте не належать конкретній сутності чи об'єкту і не мають стану.

Розглянемо основні шаблони їх використання.

- **Repositories** - Репозиторій в DDD оперує Aggregate Roots. Цей патерн дозволяє отримати Aggregate Root, а також забезпечує збереження його стану. В об’єктно-орієнтованих мовах може бути частиною ORM.
- **Application Services** забезпечують передачу запитів та команд з зовнішнього світу в модель. Утворюють тонкий шар оркестрації, куди відбувається ін’єкція інфраструктурних компонентів (реалізації репозиторіїв і тд) та викликаються методи агрегатів. Не повинні містити доменної логіки.
- **Domain Service** забезпечують оркестровку функціоналу з використанням багатьох агрегатів. Містять бізнес-логіку. Можна замінити на послідовність подій та їх обробників.
- **Factory** (Фабрики) - дозволяють створити доменний об’єкт чи агрегат за одну атомарну операцію, отримуючи всі необхідні дані, такі як параметри виклику клієнта. Не є частиною доменної моделі, але є частиною Domain Layer, адже є реалізацією бізнес-вимог застосунку.

Хід роботи:

1. Обрати ідею для технологічного бізнесу (СЕРВІСУ) на свій смак або за варіантом (в додатку)
2. Провести Event Storming сесію. Скіншот борди включити в протокол
3. Виділити Bounded contexts. Провести картографування контекстів. Отриману діаграму включити в протокол.
4. Щонайменше для 2х виділених контексів перелічити Entities, Aggregates, Value objects. Сформувати загальну мову (ubiquitous language)

5. Визначити програмні компоненти (мікросервіси). Перелічити їх та показати їх зв’язність на схемі. Відобразити їх комунікацію, а також інфраструктурні компоненти. Показати міжсервісну діаграму викликів (сіквенс діаграма)
6. В висновках пояснити, з чим розібрались в процесі проєктування, та пояснити переваги й недоліки сервіс- орієнтованого підходу для конкретної задачі.

Контрольні питання

1. В чому є важливість правильної декомпозиції системи. Концепції зв’язності та зчеплення (loose coupling/strong cohesion)

1.1. Поясніть ризики надмірної зв’язності

1.2. Поясніть ризики надмірної декомпозиції

2. Поясніть основні принципи DDD.

3. Поясніть вимоги SOLID до архітектур ПЗ

Додаток 1

1. Хмарна інформаційна система записної книжки користувача.
2. Хмарна інформаційна система обліку книжок в домашній бібліотеці.
3. Хмарна інформаційна система для реєстрації витрат палива та показників одометру для авто.
4. Хмарна інформаційна система для календаря користувача (за бажанням використовувати API Google Calendar).
5. Хмарна інформаційна система для обліку часу роботи за комп'ютером.
6. Хмарна інформаційна система для обліку часу заняття фізичними вправами.
7. Хмарна інформаційна система для обліку спожитих калорій.
8. Хмарна інформаційна система для розрахунку площі земельної ділянки (за бажанням використовуйте введення користувачем GPS координат).
9. Хмарна інформаційна система для реєстрації значення зросту дитини.
10. Хмарна інформаційна система для обліку кількості кілометрів, які пройдені на велосипеді.
11. Хмарна інформаційна система для запису шкільного домашнього завдання.
12. Хмарна інформаційна система для обліку списку необхідних справ (ToDo).
13. Хмарна інформаційна система для обрахунку графіку поливу рослин (за бажанням використовувати відкритий API метеослужби).
14. Хмарна інформаційна система для реєстрації витрат щомісячного бюджету.
15. Хмарна інформаційна система для розрахунку щоденного фізичного навантаження з урахуванням маси та зросту людини.
16. Хмарна інформаційна система для реєстрації розкладу чергування.
17. Хмарна інформаційна система для реєстрації відвідування студентами лекцій, практичних та лабораторних занять.
18. Хмарна інформаційна система для обліку часу, витраченого на дорогу до університету та з університету.

19. Хмарна інформаційна система для відстеження відправлень з адміністративною та клієнтською частиною.
20. Хмарна інформаційна система для реєстрації особистого відвідування лікаря дантиста.
21. Хмарна інформаційна система для розрахунку витрат на пальне з урахуванням рівня споживання транспортного засобу.
22. Хмарна інформаційна система для реєстрації кількості сторінок прочитаних книжок за жанрами (технічна література, художня література та інші).
23. Хмарна інформаційна система для відстеження відправлень з адміністративною та клієнтською частиною.
24. Хмарна інформаційна система для реєстрації витрат на опалення та енергоспоживання.
25. Хмарна інформаційна система для розрахунку рекомендованої кількості споживання води відносно маси людини.
26. Хмарна інформаційна система для реєстрації щоденної температури води та повітря (за бажанням використовувати API метеослужби).
27. Хмарна інформаційна система для розрахунку кількості днів до дня народження користувача.
28. Система для збереження файлів в хмарному середовищі.
29. Система для збереження та перегляду зображень в хмарному середовищі.
30. Система для збереження та відтворення аудіо інформації в хмарному середовищі.